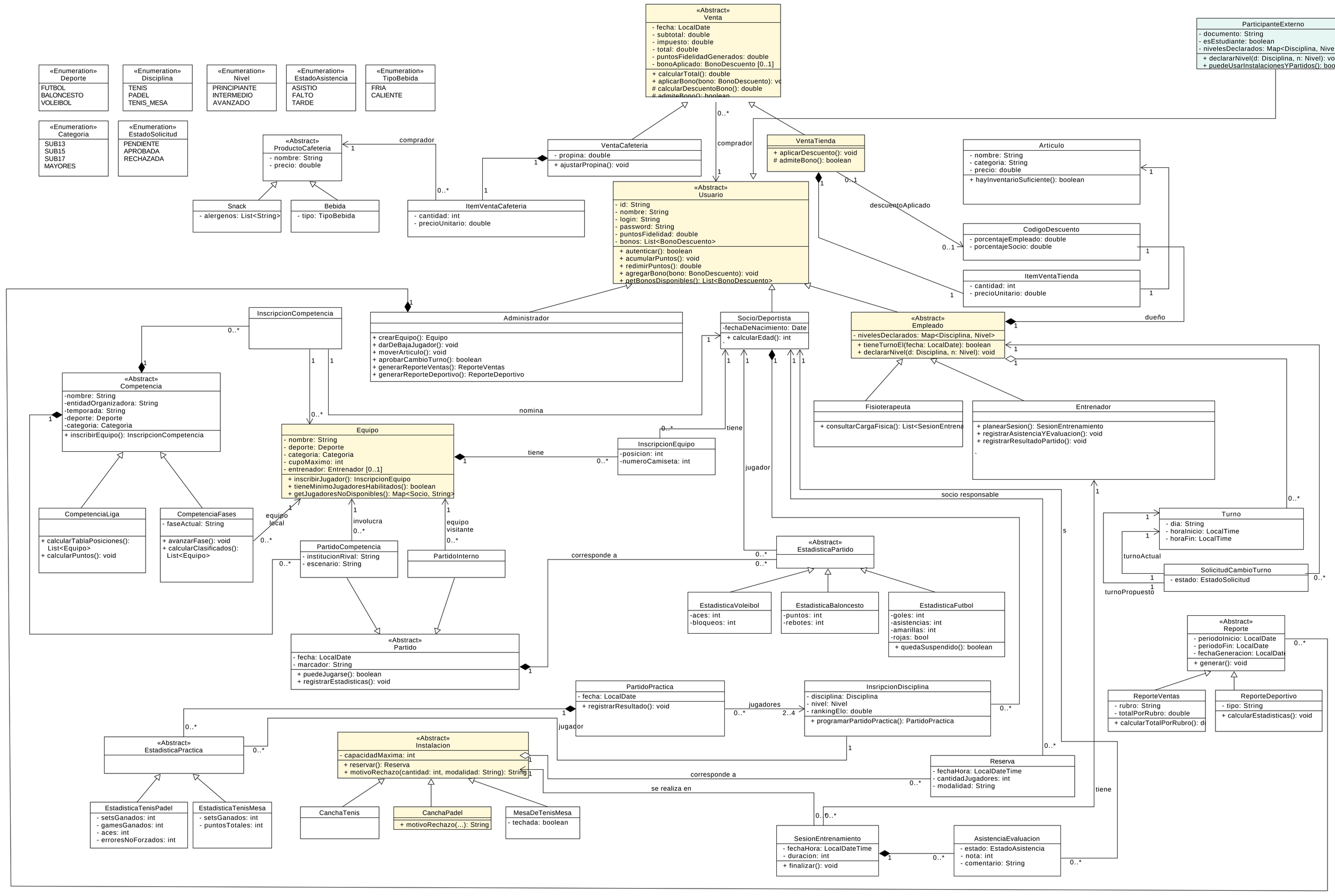
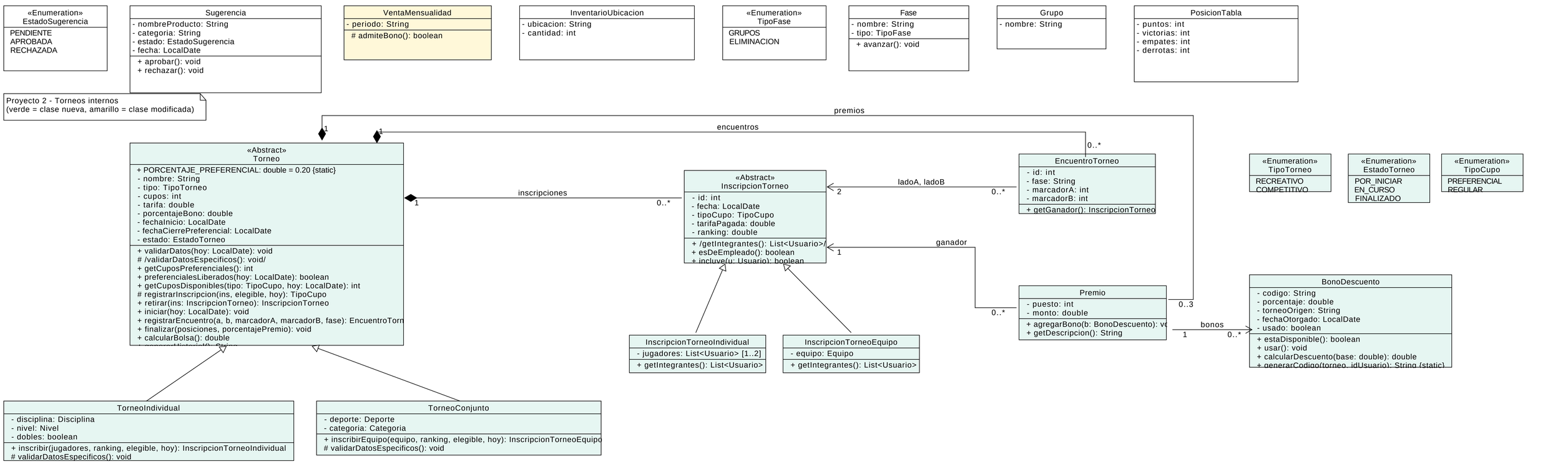

«Abstract»
Venta
- fecha: LocalDate
- subtotal: double
- impuesto: double
- total: double
- puntosFidelidadGenerados: double
- bonoAplicado: BonoDescuento [0..1]
+ calcularTotal(): double
+ aplicarBono(bono: BonoDescuento): vo
# calcularDescuentoBono(): double
# admiteBono(): boolean
ParticipanteExterno
- documento: String
- esEstudiante: boolean
- nivelesDeclarados: Map<Disciplina, Nivel>
+ declararNivel(d: Disciplina, n: Nivel): void
+ puedeUsarInstalacionesYPartidos(): boolean
«Enumeration»
Deporte
FUTBOL
BALONCESTO
VOLEIBOL
«Enumeration»
Disciplina
TENIS
PADEL
TENIS_MESA
«Enumeration»
Nivel
PRINCIPIANTE
INTERMEDIO
AVANZADO
«Enumeration»
EstadoAsistencia
ASISTIO
FALTO
TARDE
«Enumeration»
TipoBebida
FRIA
CALIENTE
«Enumeration»
Categoria
SUB13
SUB15
SUB17
MAYORES
«Enumeration»
EstadoSolicitud
PENDIENTE
APROBADA
RECHAZADA
«Abstract»
ProductoCafeteria
- nombre: String
- precio: double
comprador
1
0..*
VentaCafeteria
- propina: double
+ ajustarPropina(): void
comprador
0..*
1
VentaTienda
+ aplicarDescuento(): void
# admiteBono(): boolean
Articulo
- nombre: String
- categoria: String
- precio: double
+ hayInventarioSuficiente(): boolean
Snack
- alergenos: List<String>
Bebida
- tipo: TipoBebida
ItemVentaCafeteria
- cantidad: int
- precioUnitario: double
«Abstract»
Usuario
- id: String
- nombre: String
- login: String
- password: String
- puntosFidelidad: double
- bonos: List<BonoDescuento>
+ autenticar(): boolean
+ acumularPuntos(): void
+ redimirPuntos(): double
+ agregarBono(bono: BonoDescuento): void
+ getBonosDisponibles(): List<BonoDescuento>
descuentoAplicado
0..1
CodigoDescuento
- porcentajeEmpleado: double
- porcentajeSocio: double
ItemVentaTienda
- cantidad: int
- precioUnitario: double
dueño
InscripcionCompetencia
Administrador
+ crearEquipo(): Equipo
+ darDeBajaJugador(): void
+ moverArticulo(): void
+ aprobarCambioTurno(): boolean
+ generarReporteVentas(): ReporteVentas
+ generarReporteDeportivo(): ReporteDeportivo
Socio/Deportista
-fechaDeNacimiento: Date
+ calcularEdad(): int
«Abstract»
Empleado
- nivelesDeclarados: Map<Disciplina, Nivel>
+ tieneTurnoEl(fecha: LocalDate): boolean
+ declararNivel(d: Disciplina, n: Nivel): void
«Abstract»
Competencia
-nombre: String
-entidadOrganizadora: String
-temporada: String
-deporte: Deporte
-categoria: Categoria
+ inscribirEquipo(): InscripcionCompetencia
nomina
Fisioterapeuta
+ consultarCargaFisica(): List<SesionEntrena
Entrenador
+ planearSesion(): SesionEntrenamiento
+ registrarAsistenciaYEvaluacion(): void
+ registrarResultadoPartido(): void
Equipo
- nombre: String
- deporte: Deporte
- categoria: Categoria
- cupoMaximo: int
- entrenador: Entrenador [0..1]
+ inscribirJugador(): InscripcionEquipo
+ tieneMinimoJugadoresHabilitados(): boolean
+ getJugadoresNoDisponibles(): Map<Socio, String>
tiene
InscripcionEquipo
-posicion: int
-numeroCamiseta: int
jugador
socio responsable
Turno
- dia: String
- horaInicio: LocalTime
- horaFin: LocalTime
turnoActual
SolicitudCambioTurno
- estado: EstadoSolicitud
turnoPropuesto
CompetenciaLiga
+ calcularTablaPosiciones(): List<Equipo>
+ calcularPuntos(): void
CompetenciaFases
- faseActual: String
+ avanzarFase(): void
+ calcularClasificados(): List<Equipo>
equipo local
involucra
equipo visitante
PartidoCompetencia
- institucionRival: String
- escenario: String
PartidoInterno
corresponde a
«Abstract»
EstadisticaPartido
EstadisticaVoleibol
-aces: int
-bloqueos: int
EstadisticaBaloncesto
-puntos: int
-rebotes: int
EstadisticaFutbol
-goles: int
-asistencias: int
-amarillas: int
-rojas: bool
+ quedaSuspendido(): boolean
«Abstract»
Partido
- fecha: LocalDate
- marcador: String
+ puedeJugarse(): boolean
+ registrarEstadisticas(): void
«Abstract»
Reporte
- periodoInicio: LocalDate
- periodoFin: LocalDate
- fechaGeneracion: LocalDate
+ generar(): void
ReporteVentas
- rubro: String
- totalPorRubro: double
+ calcularTotalPorRubro(): d
ReporteDeportivo
- tipo: String
+ calcularEstadisticas(): void
PartidoPractica
- fecha: LocalDate
+ registrarResultado(): void
jugadores
InsripcionDisciplina
- disciplina: Disciplina
- nivel: Nivel
- rankingElo: double
+ programarPartidoPractica(): PartidoPractica
jugador
«Abstract»
EstadisticaPractica
«Abstract»
Instalacion
- capacidadMaxima: int
+ reservar(): Reserva
+ motivoRechazo(cantidad: int, modalidad: String): String
Reserva
- fechaHora: LocalDateTime
- cantidadJugadores: int
- modalidad: String
corresponde a
se realiza en
tiene
EstadisticaTenisPadel
- setsGanados: int
- gamesGanados: int
- aces: int
- erroresNoForzados: int
EstadisticaTenisMesa
- setsGanados: int
- puntosTotales: int
CanchaTenis
CanchaPadel
+ motivoRechazo(...): String
MesaDeTenisMesa
- techada: boolean
SesionEntrenamiento
- fechaHora: LocalDateTime
- duracion: int
+ finalizar(): void
AsistenciaEvaluacion
- estado: EstadoAsistencia
- nota: int
- comentario: String


«Enumeration»
EstadoSugerencia
PENDIENTE
APROBADA
RECHAZADA
Sugerencia
- nombreProducto: String
- categoria: String
- estado: EstadoSugerencia
- fecha: LocalDate
+ aprobar(): void
+ rechazar(): void
VentaMensualidad
- periodo: String
admiteBono(): boolean
InventarioUbicacion
- ubicacion: String
- cantidad: int
«Enumeration»
TipoFase
GRUPOS
ELIMINACION
Fase
- nombre: String
- tipo: TipoFase
+ avanzar(): void
Grupo
- nombre: String
PosicionTabla
- puntos: int
- victorias: int
- empates: int
- derrotas: int
Proyecto 2 - Torneos internos
(verde = clase nueva, amarillo = clase modificada)
premios
encuentros
«Abstract»
Torneo
+ PORCENTAJE_PREFERENCIAL: double = 0.20 {static}
- nombre: String
- tipo: TipoTorneo
- cupos: int
- tarifa: double
- porcentajeBono: double
- fechaInicio: LocalDate
- fechaCierrePreferencial: LocalDate
- estado: EstadoTorneo
+ validarDatos(hoy: LocalDate): void
# /validarDatosEspecificos(): void/
+ getCuposPreferenciales(): int
+ preferencialesLiberados(hoy: LocalDate): boolean
+ getCuposDisponibles(tipo: TipoCupo, hoy: LocalDate): int
# registrarInscripcion(ins, elegible, hoy): TipoCupo
+ retirar(ins: InscripcionTorneo): InscripcionTorneo
+ iniciar(hoy: LocalDate): void
+ registrarEncuentro(a, b, marcadorA, marcadorB, fase): EncuentroTorn
+ finalizar(posiciones, porcentajePremio): void
+ calcularBolsa(): double
inscripciones
«Abstract»
InscripcionTorneo
- id: int
- fecha: LocalDate
- tipoCupo: TipoCupo
- tarifaPagada: double
- ranking: double
+ /getIntegrantes(): List<Usuario>/
+ esDeEmpleado(): boolean
+ incluye(u: Usuario): boolean
ladoA, ladoB
EncuentroTorneo
- id: int
- fase: String
- marcadorA: int
- marcadorB: int
+ getGanador(): InscripcionTorneo
ganador
«Enumeration»
TipoTorneo
RECREATIVO
COMPETITIVO
«Enumeration»
EstadoTorneo
POR_INICIAR
EN_CURSO
FINALIZADO
«Enumeration»
TipoCupo
PREFERENCIAL
REGULAR
Premio
- puesto: int
- monto: double
+ agregarBono(b: BonoDescuento): vo
+ getDescripcion(): String
bonos
BonoDescuento
- codigo: String
- porcentaje: double
- torneoOrigen: String
- fechaOtorgado: LocalDate
- usado: boolean
+ estaDisponible(): boolean
+ usar(): void
+ calcularDescuento(base: double): double
+ generarCodigo(torneo, idUsuario): String {static}
InscripcionTorneoIndividual
- jugadores: List<Usuario> [1..2]
+ getIntegrantes(): List<Usuario>
InscripcionTorneoEquipo
- equipo: Equipo
+ getIntegrantes(): List<Usuario>
TorneoIndividual
- disciplina: Disciplina
- nivel: Nivel
- dobles: boolean
+ inscribir(jugadores, ranking, elegible, hoy): InscripcionTorneoIndividual
# validarDatosEspecificos(): void
TorneoConjunto
- deporte: Deporte
- categoria: Categoria
+ inscribirEquipo(equipo, ranking, elegible, hoy): InscripcionTorneoEquipo
# validarDatosEspecificos(): void